

BSIT 2-3

Comp 009 / Object Oriented Programming
Midterm Assignment # 1 Introduction to JavaQuestions:

- 1.) What is Java programming language? Discuss its key features & advantages.
- 2.) Explain the concepts of platform independence in Java. How does Java achieve platform independence?
- 3.) Discuss the main components of a Java program & their roles.
- 4.) What are the difference between JDK, JRE, & JVM in Java?
- 5.) Explain the difference between interpreted & compiled languages. Is Java an interpreted or compiled language?

Answers:

1.) I understand essentially in my own understanding is that Java is a high-level, object oriented programming language developed by Sun Microsystems in 1995, now maintain by Oracle. It was designed to be simple, secure & portable, with syntax influenced by C & C++ but streamlined for ease of use. Upon my own research & by research is just a couple of minutes google Java's greatest strength is that its versatility: Java can be used to build desktop applications, mobile apps, web applications, & enterprise-level system. Its key feature include object oriented principles such as encapsulation, inheritance, & polymorphism, which can make programs modular & reusable. Java also emphasizes its robustness, & security, offering strong memory management & built in safeguards against common programming errors. Furthermore, it supports multi-threading, allowing developers to execute multiple tasks simultaneously, & provide a rich Application Programming Interface (API) for networking, data handling, & graphical user interface. These Qualities make Java one of the most widely adopted programming languages worldwide.

2.) I understand platform independence just essentially means that a program written in Java can run on any operating system without modification. This feat is only achieved through the concept of byte code. When a Java program is compiled, its source code is not translated directly into machine code specific to one operating system. Instead, it is converted into byte code, a universal intermediate representation.

The Java Virtual Machine (JVM) then interprets or compiles this bytecode into native machine code for the host systems. Because JVMs exist for all major operating systems, the same Java program can run seamlessly across Windows, Linux, Mac OS, & other platforms. This principle is often summarized as "Write Once, Run Anywhere," which has been a cornerstone of Java's success.

3) The main components of a Java program is essentially a set of essential components that work together to define its structure & behavior. At the highest level, packages are used to organize related classes & prevent naming conflicts. Essentially we can see that the classes serve as blueprints that define both data (fields) & behavior (methods). These objects are instances of these classes, representing real-world entities within the program. The methods specify the actions or logic that objects can perform, & variables store the data needed for computation. The most critical component is the main method, written as `public static void main (String args)`, in which this serves as the entry point of every Java application. Without this method the program cannot execute. Together, these components provide a structured & modular way to build applications.

4) The JDK is the full toolkit for developers it includes the compiler (javac), debugger, & all tools needed to write & build Java programs. The JRE is a smaller package that contains the libraries & the JVM required to run Java applications but it can't be used to develop them. At the core is the JVM, which executes Java byte code by interpreting it or compiling it into machine code. In simple terms: JDK includes the JRE, & the JRE includes the JVM-developer's use JDK, while users rely on the JRE/JVM to run program.

5) Compiled languages like C & C++ convert source code directly into machine code before execution, making them fast but less portable. Interpreted languages like Python run code line by line through an interpreter, which increases portability but reduces speed. Java uses a hybrid model: the compiler first converts code into byte code, the JVM then interprets it or uses JIT compilation to turn it into machine code at runtime. This approach gives Java both portability & efficient performance.